

```
'''import random

class ChannelSpectrum:
    """Estima o espectro de rádio e as leituras de sinal de canais vizinhos."""
    def __init__(self, size=23, center=15):
        self.channels = [0.0] * size
        self.center = center
        # Cria um sinal gaussiano simplificado centrado no canal center
        for i in range(size):
            dist = abs(i - center)
            # Sinal diminui com a distância
            if dist == 0:
                self.channels[i] = 1.0 # Canal central
            elif dist <= 2:
                self.channels[i] = 0.5 # Canal adjacente
            elif dist <= 4:
                self.channels[i] = 0.2 # Canal distante

    def get_signal_at(self, channel_index):
        if 0 <= channel_index < len(self.channels):
            return self.channels[channel_index]
        return 0.0

class ChannelManager:
    """Gerencia a lista de canais ativos e bloqueados (blacklist)."""
    def __init__(self):
        self.active_channels = set() # Canais em uso (desbloqueados)
        self.blacklist = set() # Canais com falhas (bloqueados)
        self.history = [] # Histórico de leituras ('new record')

    def add_new_record(self, channel):
        """Simula a chegada de um 'new record' (Case A)."""
        record = {'channel': channel, 'timestamp': len(self.history)}
        self.history.insert(0, record) # O mais novo é o primeiro
        return record

    def get_unlocked_channels(self):
        # Na figura, são os canais verdes A e A1
        if len(self.history) < 1:
            return None, None

        A = self.history[0]['channel'] # O mais recente
        A1 = None

        # Encontra o registro anterior na lista (A1)
        for record in self.history[1:]:
            if record['channel'] != A:
                A1 = record['channel']
                break

        return A, A1

class ChannelAlgorithmSimulator:
    """Implementa as lógicas dos algoritmos e decisão da figura."""
    def __init__(self):
        self.spectrum = ChannelSpectrum()
        self.manager = ChannelManager()
        # Limiar de interferência para bloqueio (ex: > 30% de sinal)
        self.interference_threshold = 0.3

    def run(self, new_channel_index):
        print(f"n--- Processando novo registro no canal: {new_channel_index} ---")

        # --- CASE A: 'New Record' Chega ---
        # Representado pelo gráfico superior e primeira linha de Case A
        new_record = self.manager.add_new_record(new_channel_index)

        # --- Identificar Canais Desbloqueados (A e A1) ---
        # Conforme a legenda: A é o novo sinal, A1 é o registro anterior
        channel_A, channel_A1 = self.manager.get_unlocked_channels()
        print(f"Legenda -> Canal A (Atual): {channel_A}, Canal A1 (Anterior): {channel_A1}")

        # Se for o primeiro registro, não há intervalo para analisar
        if channel_A1 is None:
            print("Nenhum intervalo para analisar. Canal A adicionado como ativo.")
            self.manager.active_channels.add(channel_A)
            return

        # --- Identificar o Intervalo Analisado ---
        # Conforme definição da figura: entre dois canais ativos (A e A1)
        interval = list(range(min(channel_A, channel_A1), max(channel_A, channel_A1) + 1))
        print(f"Intervalo Analisado -> {interval}")

        estimated_signals = {}

        # --- Algoritmo I (Estimativa do Primeiro Canal do Intervalo) ---
        # Estima a interferência no canal logo após o início do intervalo
        first_channel = interval[1] # B na legenda
        first_signal = self.spectrum.get_signal_at(first_channel)
        estimated_signals[first_channel] = ('B', first_signal)

        print(f"Algoritmo I -> Canal B ({first_channel}) Estima Nível: {first_signal:.2f}")

        # --- Algoritmo II (Estimativa das Extremidades do Intervalo) ---
        # Estima interferência nas pontas do intervalo (A e A1 já ativos, mas re-estimados)
        signal_A = self.spectrum.get_signal_at(channel_A)
        signal_A1 = self.spectrum.get_signal_at(channel_A1)
        estimated_signals[channel_A] = ('A', signal_A)
        estimated_signals[channel_A1] = ('A1', signal_A1)
        print(f"Algoritmo II -> Estima Níveis: A ({channel_A})={signal_A:.2f}, A1 ({channel_A1})={signal_A1:.2f}")

        # Se houver canais intermediários suficientes (Case B e C)
        if len(interval) >= 4:
            # B1 na legenda: último canal antes do fim do intervalo
            last_channel = interval[-2]
            last_signal = self.spectrum.get_signal_at(last_channel)
            estimated_signals[last_channel] = ('B1', last_signal)
            print(f"Algoritmo II -> Canal B1 ({last_channel}) Estima Nível: {last_signal:.2f}")

        # --- Algoritmo III (Estimativa de Canais Intermediários Adicionais) ---
        # Só executa se houver espaço para C e C1 (Case C: intervalo de 6 canais)
        if len(interval) >= 6:
            # C na legenda: segundo canal do intervalo
            second_channel = interval[2]
            # C1 na legenda: penúltimo canal do intervalo
            penultimate_channel = interval[-3]

            signal_C = self.spectrum.get_signal_at(second_channel)
            signal_C1 = self.spectrum.get_signal_at(penultimate_channel)

            estimated_signals[second_channel] = ('C', signal_C)
            estimated_signals[penultimate_channel] = ('C1', signal_C1)
            print(f"Algoritmo III -> Canais C ({second_channel})={signal_C:.2f}, C1 ({penultimate_channel})={signal_C1:.2f}")

        # --- Algoritmo de Decisão (Decision Algorithm) ---
        # Decide bloquear ou desbloquear com base nos resultados estimados
        print(f"n--- Algoritmo de Decisão ---")

        # Limpa o estado atual dos canais do intervalo (exceto A e A1 que são fixos)
        for ch in interval:
            if ch != channel_A and ch != channel_A1:
                if ch in self.manager.active_channels: self.manager.active_channels.remove(ch)
                if ch in self.manager.blacklist: self.manager.blacklist.remove(ch)

        final_active = {channel_A, channel_A1}
        final_blocked = set()

        for ch, (label, signal) in estimated_signals.items():
            # A e A1 são mantidos ativos independentemente da re-estimativa
            if label == 'A' or label == 'A1':
                continue

            # Decisão de bloqueio baseada no limiar
            if signal > self.interference_threshold:
                final_blocked.add(ch)
                print(f" Bloquear Canal {ch} (Legenda {label}): Sinal {signal:.2f} > Limiar {self.interference_threshold}")
            else:
                final_active.add(ch)
                print(f" Desbloquear Canal {ch} (Legenda {label}): Sinal {signal:.2f} <= Limiar {self.interference_threshold}")

        # Atualiza o estado global no gerenciador
        self.manager.active_channels.update(final_active)
        self.manager.blacklist.update(final_blocked)

        # --- Resumo Final do Estado da Rede ---
        print(f"n--- Estado Final da Rede conforme a Figura ---")
        unlocked_viz = "Verde"
        blocked_viz = "Amarelo"

        full_resumo = []
        for ch in range(min(interval), max(interval) + 1):
            if ch in self.manager.active_channels:
                label = estimated_signals.get(ch, ('A/A1', ''))[0]
                full_resumo.append(f"C{ch}({unlocked_viz}-{label})")
            elif ch in self.manager.blacklist:
                label = estimated_signals.get(ch, ('', ''))[0]
                full_resumo.append(f"C{ch}({blocked_viz}-{label})")
            else:
                full_resumo.append(f"C{ch}(Vazio)")

        print(" | ".join(full_resumo))
```